GUIA DE REPASO

Modulo 2

Extraccion de datos
Clases 5 a 8

Una ayuda memoria para trabajar con Pandas, archivos CSV, SQL, SQLite y exploracion inicial de datos.

Como usar esta guia

Cada clase resume la idea principal, los comandos que vas a usar y ejemplos breves para adaptar en ejercicios. El objetivo es tener una referencia rapida antes de consultar documentacion.

Clase 5 - Introduccion a Pandas

Pandas es una libreria para trabajar con datos tabulares: filas y columnas, como una planilla. Permite responder preguntas sobre una tabla sin programar bucles manuales para cada operacion.

Preparar Pandas

```
pip install pandas

import pandas as pd
```

Comando	Funcion
pip install pandas	Instala Pandas si el entorno no lo trae incluido.
import pandas as pd	Importa la libreria con el alias estandar pd.
pd.Series([...])	Crea una columna individual con indice.
pd.DataFrame([...])	Crea una tabla a partir de filas o diccionarios.

Series y operaciones vectorizadas

```
precios = pd.Series([12000, 8500, 34000])
precios_con_iva = precios * 1.19
```

La multiplicacion se aplica a cada elemento de la Series. Esto evita escribir un for para transformaciones simples sobre una columna.

DataFrames y archivos CSV

```
df = pd.read_csv("ventas.csv")
print(df.head())
```

Elemento	Funcion
df.shape	Devuelve una tupla: cantidad de filas y columnas.
df.columns	Muestra los nombres de las columnas.
df.dtypes	Muestra el tipo de dato de cada columna.
df.head(n)	Muestra las primeras n filas.
df["precio"]	Selecciona una columna y devuelve una Series.
df[["producto", "precio"]]	Selecciona varias columnas.
df.iloc[0:3]	Selecciona filas por posicion.

Recorda: si pd.read_csv() no encuentra el archivo, revisa la ruta. Si no esta en la misma carpeta, indica la ruta correcta.

Clase 6 - Introduccion a SQL

SQL es un lenguaje para consultar datos directamente en una base de datos. En lugar de descargar todo, pedis solo las columnas y filas que necesitas.

Consulta basica

```
SELECT * FROM productos;

SELECT nombre, precio FROM productos;
```

Clausula	Funcion
SELECT	Indica las columnas que queres obtener.
*	Representa todas las columnas.
FROM	Indica la tabla de origen.
;	Cierra la instruccion SQL.

Filtrar y limitar

```
SELECT * FROM productos WHERE categoria = 'tech';

SELECT * FROM productos WHERE categoria = 'tech' AND precio > 50000;

SELECT * FROM productos LIMIT 3;
```

Elemento	Funcion
WHERE	Filtra filas segun una condicion.
AND / OR	Combina condiciones.
=, !=, >, <, >=, <=	Compara valores. En SQL igualdad se escribe con un solo =.
LIMIT	Restringe la cantidad de filas devueltas.

Tablas relacionadas y JOIN

```
SELECT clientes.nombre, pedidos.producto, pedidos.monto
FROM pedidos
JOIN clientes ON pedidos.cliente_id = clientes.id;
```

Las bases relacionales separan la informacion en tablas para no duplicarla. Una clave, como id o cliente_id, conecta las filas. JOIN combina datos de tablas relacionadas a partir de esa clave.

Orden tipico de una consulta: SELECT, FROM, JOIN, WHERE, LIMIT.

Clase 7 - SQLite con Python

SQLite guarda una base de datos en un archivo local. Python puede conectarse a ella y Pandas puede traer el resultado de una consulta SQL directo a un DataFrame.

Flujo de trabajo

```
import sqlite3
import pandas as pd

conexion = sqlite3.connect("ventas.db")
df = pd.read_sql("SELECT * FROM productos", conexion)
conexion.close()
```

Comando	Funcion
import sqlite3	Importa el modulo incluido en Python para SQLite.
sqlite3.connect("archivo.db")	Abre una base existente o crea el archivo si no existe.
conexion.cursor()	Crea un cursor para ejecutar SQL manualmente.
cursor.execute(sql)	Ejecuta instrucciones SQL como CREATE o INSERT.
conexion.commit()	Confirma los cambios realizados en la base.
pd.read_sql(sql, conexion)	Ejecuta una consulta y devuelve su resultado como DataFrame.
conexion.close()	Cierra la conexion al terminar.

Crear una tabla y consultar

```
cursor.execute("""
CREATE TABLE IF NOT EXISTS productos (
id INTEGER PRIMARY KEY,
nombre TEXT,
precio REAL
)
""")
conexion.commit()
```

```
df_tech = pd.read_sql(
"SELECT * FROM productos WHERE categoria = 'tech'",
conexion
)
```

Explorar una base que ya existe

```
sql_tablas = "SELECT name FROM sqlite_master WHERE type='table'"
tablas = pd.read_sql(sql_tablas, conexion)
estructura = pd.read_sql("PRAGMA table_info(productos)", conexion)
```

Recorda: mientras uses la conexion podés hacer varias consultas. Cerrala solo cuando terminaste de leer o modificar la base.

Clase 8 - Exploracion inicial de datos

Antes de sacar conclusiones, hay que conocer el dataset: tamaño, columnas, tipos, valores faltantes, rangos numericos y categorias. Esta etapa se llama EDA: Exploratory Data Analysis.

Inspeccion inicial de un DataFrame

```
print(df.shape)
print(df.head())
print(df.tail())
df.info()
df.describe()
```

Herramienta	Para que sirve
df.shape	Cantidad de filas y columnas.
df.head(n)	Muestra las primeras n filas.
df.tail(n)	Muestra las ultimas n filas.
df.info()	Muestra estructura, valores no nulos y tipos de datos.
df.describe()	Calcula estadisticas de columnas numericas.
df["categoria"].value_counts()	Cuenta cada categoria.
value_counts(normalize=True)	Devuelve proporciones por categoria.

Estadisticas descriptivas

describe() resume count, mean, std, min, cuartiles 25%, 50% y 75%, y max. Sirve para detectar rapidamente valores inesperados, por ejemplo una edad maxima imposible.

Primeros graficos

```
import matplotlib.pyplot as plt

df['categoria'].value_counts().plot(kind='bar')
plt.title('Productos por categoria')
plt.xlabel('Categoria')
plt.ylabel('Cantidad')
plt.show()
```

Grafico	Cuando usarlo
Barras: plot(kind='bar')	Comparar cantidades entre categorias.
Histograma: plot(kind='hist')	Ver como se distribuye una variable numerica.
plt.title(), xlabel(), ylabel()	Agregar contexto para que el grafico se entienda solo.
plt.show()	Mostrar el grafico.

Recorda: explora primero y analiza despues. head(), tail(), info() y describe() suelen ser las primeras lineas de cualquier notebook con un dataset nuevo.

Repaso final del Modulo 2

El Modulo 2 enseña distintas formas de obtener y conocer datos. Empezas con archivos CSV, avanzas a consultas SQL y SQLite, y terminas explorando lo que recibiste antes de analizarlo en profundidad.

Mapa de decisiones rapido

Si necesitas...	Usa...
Cargar una tabla desde un CSV	pd.read_csv("archivo.csv")
Crear una tabla desde diccionarios	pd.DataFrame(lista_de_diccionarios)
Ver columnas y tipos	df.columns, df.dtypes o df.info()
Seleccionar una columna	df["columna"]
Seleccionar filas por posicion	df.iloc[...]
Pedir datos a una base	SELECT ... FROM ...
Filtrar en SQL	WHERE condicion
Unir tablas relacionadas	JOIN ... ON ...
Conectar Python a SQLite	sqlite3.connect("archivo.db")
Traer SQL a Pandas	pd.read_sql(sql, conexion)
Explorar tamaño y muestra	shape, head(), tail()
Resumir numeros y categorias	describe() y value_counts()
Crear un grafico basico	plot(kind=...) y plt.show()

Checklist cuando recibis datos nuevos

1. Identifica de donde vienen: CSV, base de datos u otra fuente.
2. Carga solo lo necesario.
3. Revisa shape, head(), tail() e info().
4. Confirma nombres y tipos de columnas.
5. Busca valores faltantes o rangos extraños con describe().
6. Cuenta categorias con value_counts().
7. Cierra la conexion SQLite al terminar.

Fin de la guia. Usa estos modelos como punto de partida y adapta los nombres de archivos, tablas y columnas a cada ejercicio.